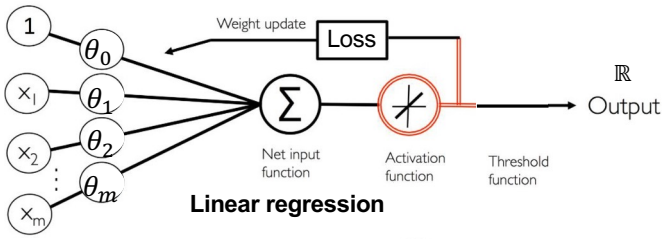
Agenda



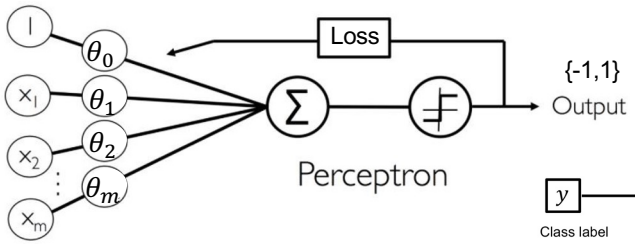
- Repetition
  - Perceptron, Adaline, Linear regression and Logistic regression
    - Models
    - Activation functions
    - Loss functions
    - Gradients and weight/parameter updates
- Logistic regression code example
- Newton´s method for weight/parameter optimisation
- Start of the Tour of Classifiers in Scikit-learn
- Quizzes

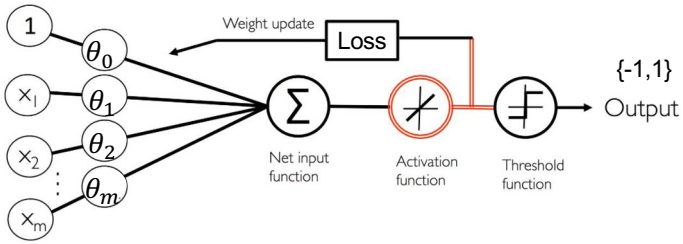
Recap

	Linear regression	Adaline	Perceptron	Logistic regression
Type	Regression	Classification	Classification	Classification
Model	Linear	Linear	Linear	Linear

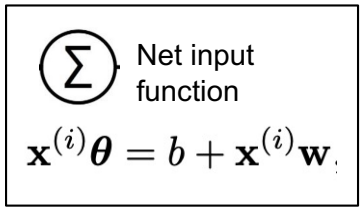


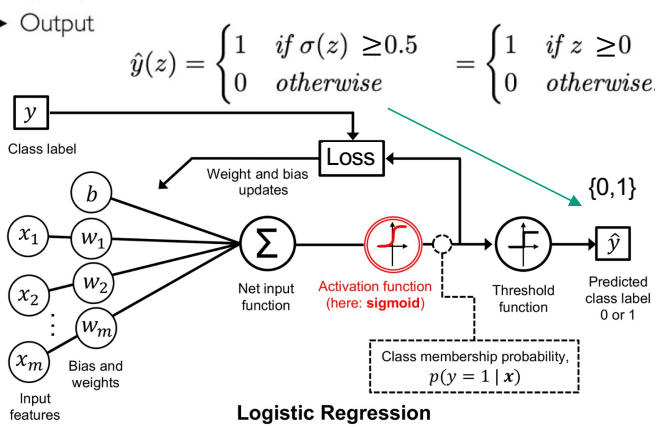
Linear regression





Adaptive Linear Neuron (Adaline)





Logistic Regression

Parameters (Linear Model)

$$\mathbf{x}^{(i)}\boldsymbol{\theta} = b + \mathbf{x}^{(i)}\mathbf{w}$$ (equals the net input *z*, which is used by all four methods)

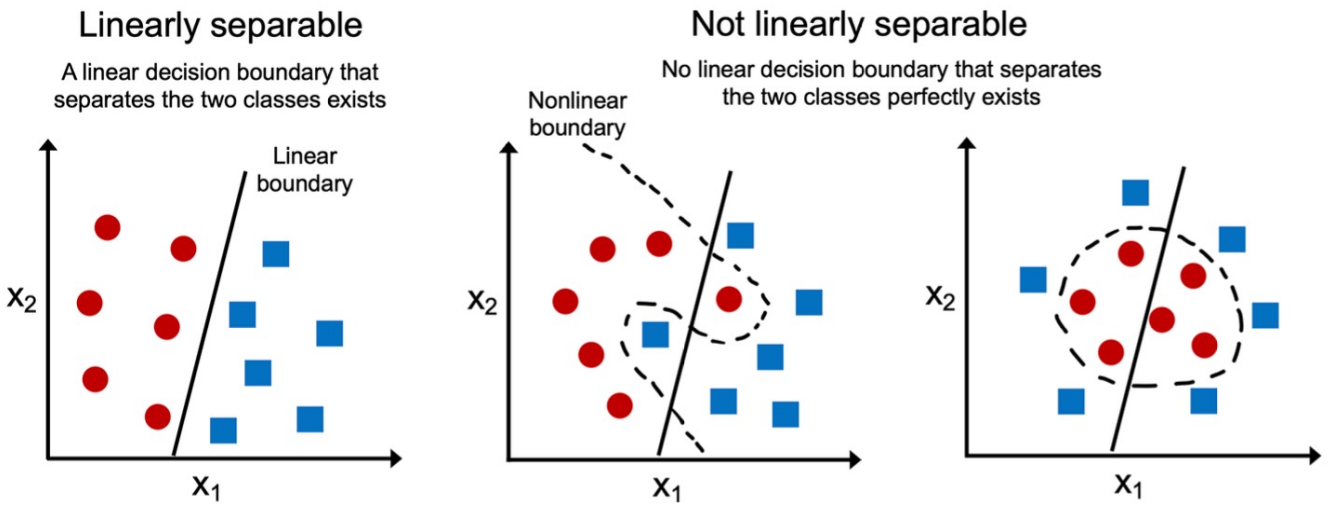
$$\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_m \end{bmatrix} := \begin{bmatrix} b \\ \mathbf{w} \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_m \end{bmatrix}, \quad b = \theta_0 \quad \text{(or } w_0\text{)}$$

(all) parameters

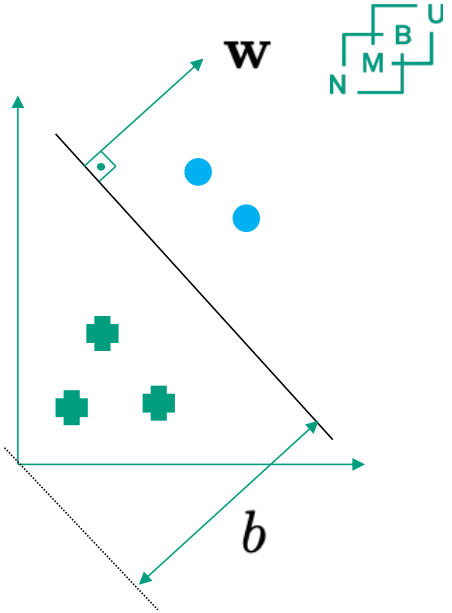
“weights” “normal”

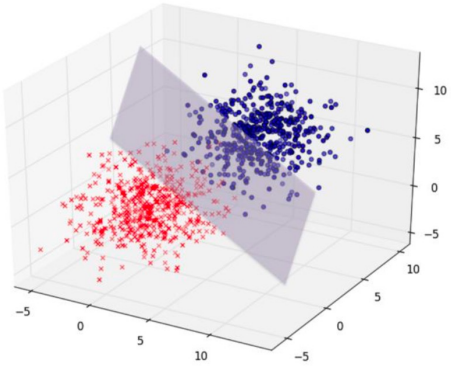
“bias” “offset”

Linearly separable



(From the book Machine Learning with Python by S. Raschka)





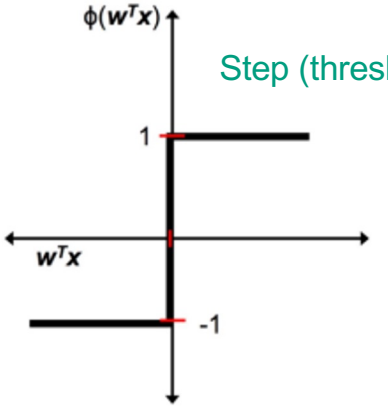


	Linear regression	Adaline	Perceptron	Logistic regression
Type	Regression	Classification	Classification	Classification
Model	Linear	Linear	Linear	Linear
Loss	Squared Loss	Squared Loss	“Hinge” Loss	Log-Loss
	$L(\boldsymbol{\theta}) = \sum_{i=1}^{n} \left(y^{(i)} - \mathbf{x}^{(i)}\boldsymbol{\theta} \right)^2$	$L(\boldsymbol{\theta}) = \sum_{i=1}^{n} \left(y^{(i)} - \mathbf{x}^{(i)}\boldsymbol{\theta} \right)^2$	$L(\boldsymbol{\theta}) = \max(0, -y\mathbf{x}^{(i)}\boldsymbol{\theta})$	$L(\boldsymbol{\theta}) = -\sum_{i=1}^{n} \left[y^{(i)} \log(\sigma(z)) + (1 - y^{(i)}) \log(1 - \sigma(z)) \right]$
Activation function	Identity	Identity	Step	Sigmoid
Solver	GD, Least Squares	GD, Least Squares	GD	GD, Newton

Activation functions

Perceptron

$$\phi(z) = \begin{cases} 1 & if\ z \geq 0 \\ -1 & otherwise \end{cases}$$

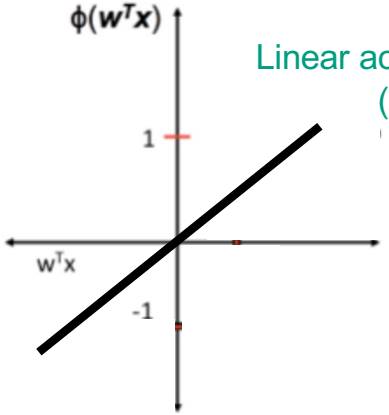


Step (threshold) function

$$z = w_0 x_0 + w_1 x_1 + w_2 x_2 + \cdots + w_m x_m$$

Adaline and linear regression

$$\phi(z) = \phi(\boldsymbol{w}^T \boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x}$$



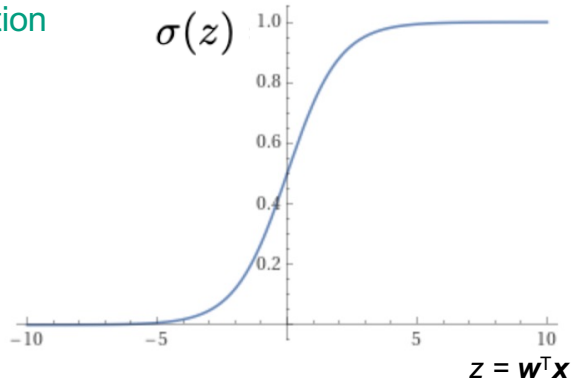
Linear activation function (identity)

Logistic regression

$$p = \sigma(z) = \frac{1}{1 + \exp(-z)}$$

$$\text{logit}(p) := \log \frac{p}{1-p} = z = \boldsymbol{w}^T \boldsymbol{x}$$

Sigmoid/logistic activation function



$$\text{logit} : (0,1) \to \mathbb{R}, \quad \sigma : \mathbb{R} \to (0,1)$$

Gradients and weight updates

Adaline and linear regression

$$J(\boldsymbol{w}) = \frac{1}{2}\sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right)^2$$

$$\frac{\partial J}{\partial w_j} = -\sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right) x_j^{(i)}$$

$$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right) x_j^{(i)}$$

Logistic regression

$$L(\boldsymbol{\theta}) = -\sum_{i=1}^{n} \left[y^{(i)} \log(\sigma(z)) + (1 - y^{(i)}) \log(1 - \sigma(z))\right]$$

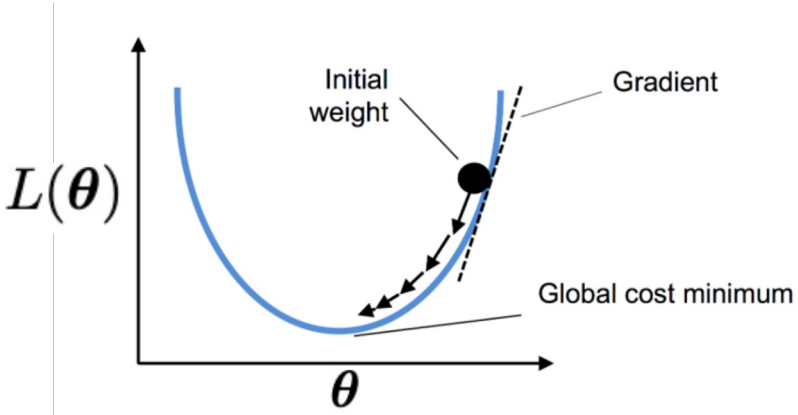
$$\frac{\partial L(\boldsymbol{\theta})}{\partial \theta_j} = -\sum_{i=1}^{n} \left(x_j^{(i)} \left[(y^{(i)} - \sigma(\mathbf{x}^{(i)}\boldsymbol{\theta}))\right]\right) = 0$$

$$\Delta \boldsymbol{\theta}_j = -\eta \frac{\partial L(\boldsymbol{\theta})}{\partial \theta_j} = \eta \sum_i \left(y^{(i)} - \sigma\left(z^{(i)}\right)\right) x_j^{(i)}$$

Goal: Maximize likelihood = Minimize negative log-likelihood

(Batch) Gradient descent

Need to choose a learning rate



$$\boldsymbol{\theta}^{n+1} = \boldsymbol{\theta} - \eta \frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}.$$

Need to adapt the learning rate

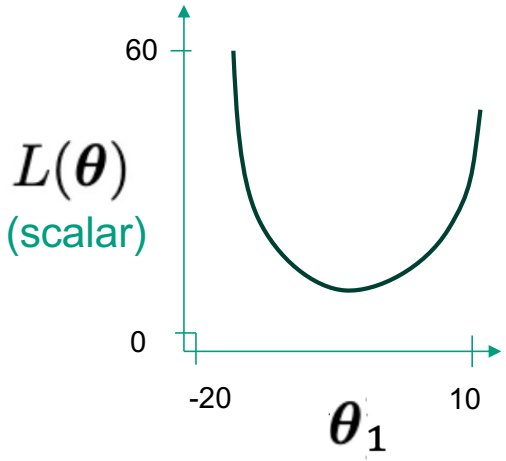
There is a way to take larger steps!

Let's look at "Newton's method"

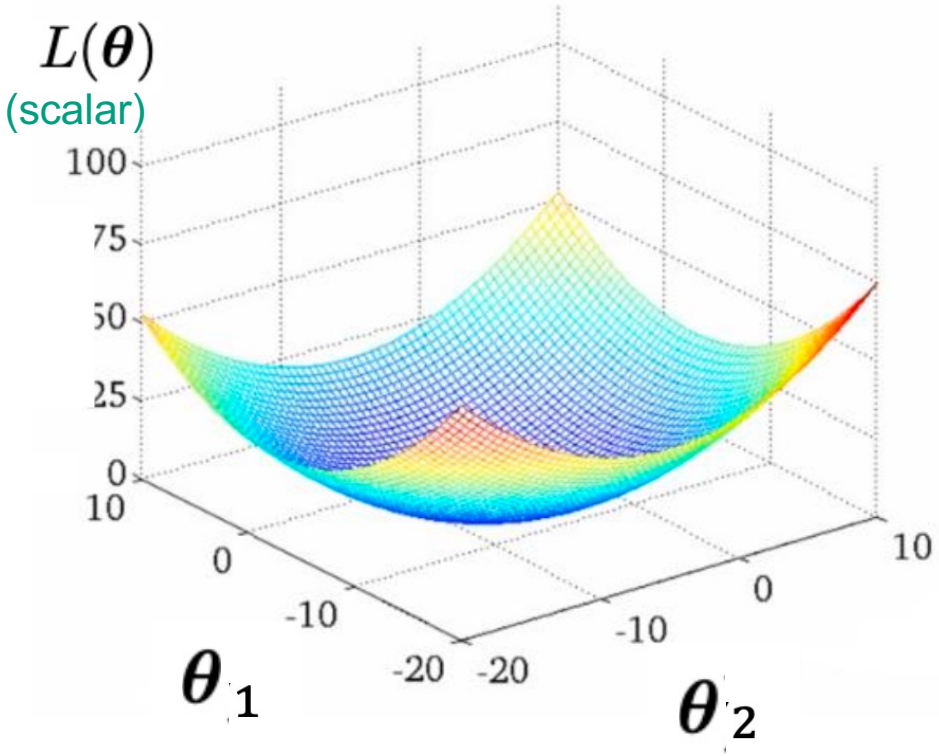
Newton’s method

Loss function

1D (1 feature)



2D (2 features)



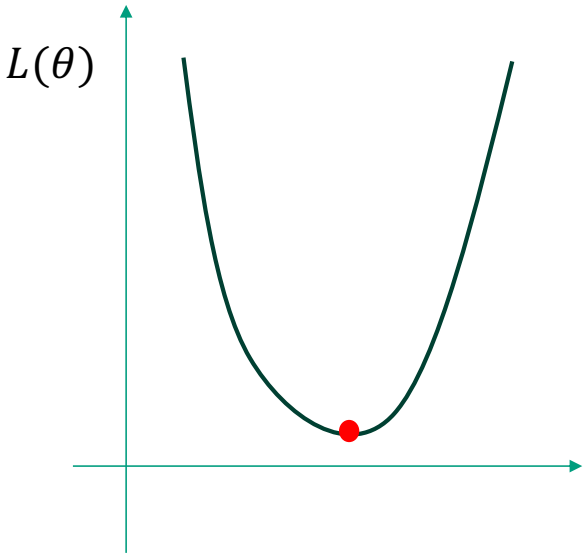
Gradient descent

Finds the minimum of a function

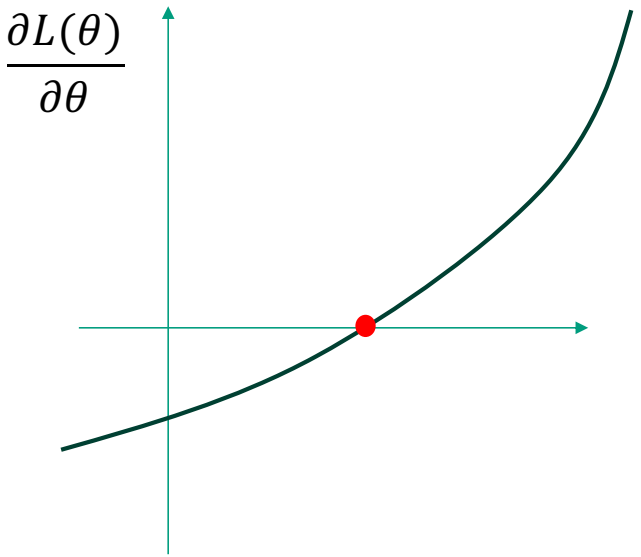
Newton's method

Finds the root (zero-point) of a function

Two different algorithms, different goals!

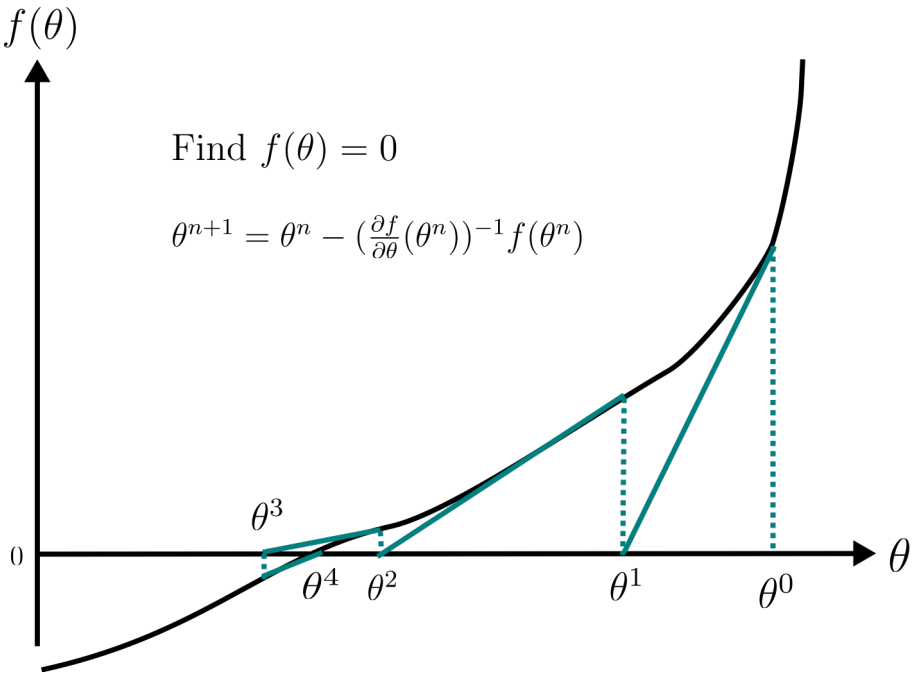


Connection: The **function derivative** is **zero** at the **minimum**!



Newton's method

Find's the roots of a function (where the function is zero)



$$\frac{\partial f(\theta)}{\partial \theta}(\theta^n) := f'(\theta^n) = \frac{f(\theta^n)}{\theta^n - \theta^{n+1}} := \frac{f(\theta^n)}{-\Delta\theta}$$

$$\theta^{n+1} = \theta^n - \frac{f(\theta^n)}{f'(\theta^n)}$$

Newton's method

For **vector-valued functions** (function outputs a vector):

Define the Jacobian matrix: $\mathbf{J}_F := \frac{\partial \mathbf{F}(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$

Use matrix inverse in the iteration rule

$$\boldsymbol{\theta}^{n+1} = \boldsymbol{\theta}^n - \mathbf{J}(\boldsymbol{\theta}^n)^{-1}\mathbf{F}(\boldsymbol{\theta}^n)$$

Remark: The function and Jacobian are evaluated at $\boldsymbol{\theta}^n$

Newton's method

In the optimization case, we want to find the root of the loss function derivative

$$\frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbf{0}$$

The loss function derivative is a **vector-valued function**.

For Newton's method we need the **Jacobian of the loss function derivative**

→ This is the second derivative of the loss function (**derivative of derivative**)

→ We call it the Hessian matrix of the loss function

$$\mathbf{H}(\boldsymbol{\theta}) := \frac{\partial^2 L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}} \quad \text{with entries} \quad H_{ij} = \frac{\partial^2 L(\boldsymbol{\theta})}{\partial \theta_i \partial \theta_j}$$

Newton's method

Therefore, in our case

$$\mathbf{F}(\boldsymbol{\theta}) = \frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbf{0} \qquad \text{and} \qquad \mathbf{J}_F = \frac{\partial^2 L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}}$$

We find the root of the loss function derivative as

$$\begin{aligned}
\boldsymbol{\theta}^{n+1} &= \boldsymbol{\theta}^n - \mathbf{J}(\boldsymbol{\theta}^n)^{-1}\mathbf{F}(\boldsymbol{\theta}^n) \\
&= \boldsymbol{\theta}^n - \mathbf{H}(\boldsymbol{\theta}^n)^{-1}\frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}^n) \\
&= \boldsymbol{\theta}^n - \left(\frac{\partial^2 L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}}\right)^{-1}\frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}^n)
\end{aligned}$$

Newton's method (some facts)

- In the vicinity of the root, Newton's method converges quadratically
  -> means that the number of significant digits doubles in every step
- Suitable for convex optimization problems by finding the root of the loss derivative
- Usually converges in **much less iterations** than gradient descent
- **But: Higher memory cost $O(n^2)$ and higher cost per step (need to invert the Hessian)**
- Robust with respect to scaling of the data
- No need to choose a learning rate, the step size is part of the algorithm (although more sophisticated variations exist)

Newton's method (extra fact)

For linear problems, Newton's method converges in a single step

Linear regression $$\frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\boldsymbol{\theta}$$

$$\mathbf{H}(\boldsymbol{\theta}) = \frac{\partial}{\partial \boldsymbol{\theta}}\left(\frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}\right) = \frac{\partial^2 L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}\partial \boldsymbol{\theta}} = 2\mathbf{X}^T\mathbf{X} \qquad \boldsymbol{\theta}^{n+1} = \boldsymbol{\theta}^n - \mathbf{H}(\boldsymbol{\theta}^n)^{-1}\frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta}^n)$$

First iteration:

$$\begin{aligned}\boldsymbol{\theta}^1 &= \boldsymbol{\theta}^0 - (2\mathbf{X}^T\mathbf{X})^{-1}(-2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\boldsymbol{\theta}^0) \\ &= \boldsymbol{\theta}^0 + (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} - \boldsymbol{\theta}^0 \\ &= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}.\end{aligned}$$

Least squares solution

Newton's method in practice

More sophisticated versions avoiding the computation of the Hessian called "Quasi-Newton schemes":

→ L-BFGS (Limited-memory Broyden–Fletcher–Goldfarb–Shanno)

(default solver in scikit-learn (1.4.0) for `LogisticRegression`)

Newton's method example

`newton_vs_gradientdescent.ipynb`